

Cortés Bolaños Angel David - 423066940

Reporte Laboratorio S5

Mexi Banco con Docker Compose.

Salida del comando `docker compose ps` que confirma que tanto el servicio de la base de datos (db) como el de la aplicación (app) lograron inicializarse y se encuentran en estado *healthy*.

```
#19 resolving provenance for metadata file
#19 DONE 0.1s
[+] up 5/5
✓ Image mexi-banco-app          Built          61.0s
✓ Network mexibanco             Created        0.1s
✓ Volume mexi-banco_mexibanco_datos Created        0.0s
✓ Container mexi-banco-db-1     Healthy       6.4s
✓ Container mexi-banco-app-1    Started       6.5s

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco ((v05.1))
$
```

```
angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco ((v05.1))
$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED      STATUS              PORTS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app        4 minutes ago Up 4 minutes (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1    postgres:17-alpine  "docker-entrypoint.s..." db         4 minutes ago Up 4 minutes (healthy) 5432/tcp
```

Probando endpoints:

Captura de la ejecución de endpoints como son la creación de la cuenta, la prueba de idempotencia, ejecución exitosa de creación de cuentas y verificación de saldos tras una transferencia interna, confirmando que la lógica transaccional del monolito funciona y los saldos cuadran.

The screenshot shows a REST client interface with a sidebar on the left containing a collection named 'Mexi Banco v05.1'. The main panel displays a POST request to 'POST Abrir cuenta origen (1000)' with the URL 'POST {{baseUrl}}/cuentas'. The response is a 201 Created status with a JSON body:

```
{  "id": 1,  "clabe": "002180790376981510",  "titular": "Ana",  "saldo": 1000}
```

Angel Cortes's Workspace

Mexi Banco v05.1 > 2 · Transferencia interna > Consultar origen (800)

GET {{baseUrl}} /cuentas/ {{origen}}

Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

200 OK · 18 ms · 232 B

Save Response

JSON

```
1 {
2   "id": 1,
3   "clabe": "002180790376981510",
4   "titular": "Ana",
5   "saldo": 800.00
6 }
```

Angel Cortes's Workspace

Consul · GET Consul · POST SPEI · POST SPEI · GET Consul · GET Estado · GET Avisos

Mexi Banco v05.1 > 4 · El monolito por dentro > Avisos del origen

GET {{baseUrl}} /notificaciones?clabe= {{origen}}

Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ clabe	{{origen}}			
Key	Value	Description		

Body

200 OK · 65 ms · 453 B

Save Response

JSON

```
1 [
2   {
3     "id": 3,
4     "clabeCuenta": "002180790376981510",
5     "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
6     "fecha": "2026-09-25T22:57:54.037691Z"
7   },
8   {
9     "id": 1,
10    "clabeCuenta": "002180790376981510",
11    "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
12    "fecha": "2026-09-25T22:57:09.880749Z"
13  }
14 ]
```

Después de compose down y compose up -d

Prueba de persistencia exitosa. Tras apagar los contenedores de forma estándar y volver a levantarlos, la consulta demuestra que la cuenta origen y su saldo se mantuvieron intactos gracias al volumen mapeado en Docker

The screenshot shows the Postman interface with a collection named 'Mexi Banco v05.1'. The request is a GET call to the endpoint `{{baseUrl}}/cuentas/{{origen}}`. The response is a 200 OK status with a response time of 467 ms. The body of the response is a JSON object representing an account.

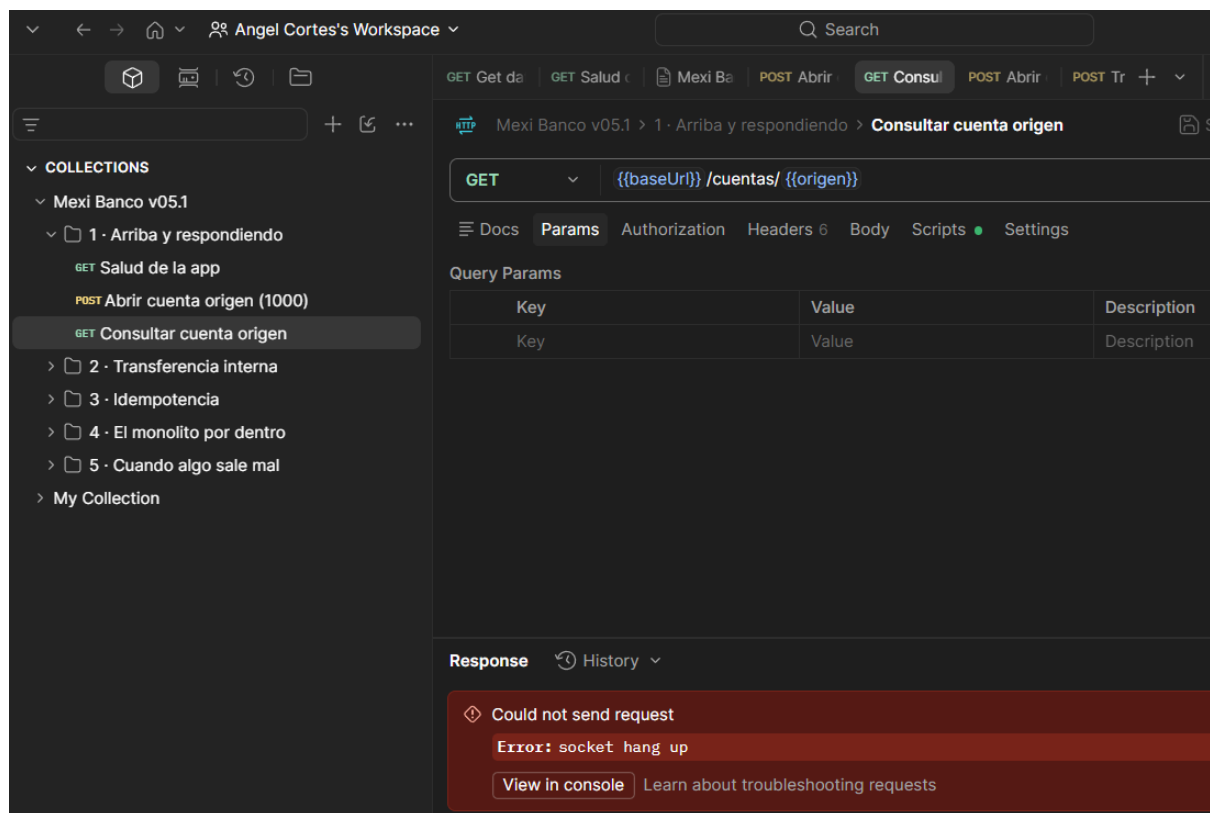
Query Params

Key	Value
Key	Value

Body

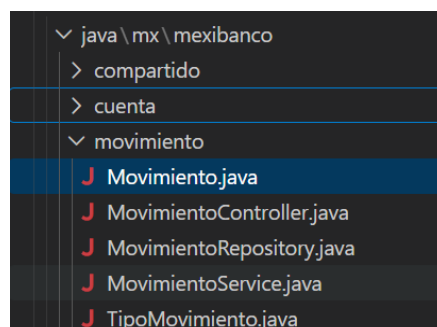
```
{  
  "id": 1,  
  "clabe": "002180790376981510",  
  "titular": "Ana",  
  "saldo": 750.00  
}
```

Con la eliminación del volumen de PostgreSQL al usar -v, el sistema inicia una base de datos completamente limpia. Como evidencia, la consulta a la cuenta origen que antes existía ahora devuelve un código de error.



Chequeo del código

Como se observa en el explorador de archivos, el código corresponde a una arquitectura monolítica donde todos los dominios del banco conviven dentro del mismo proyecto bajo la ruta `src/main/java/mx/mexibanco`. Se pueden identificar claramente las carpetas de los distintos módulos: cuenta, movimiento, notificacion, spei y transferencia. Y como se menciona las instrucciones de la práctica, cada uno de estos módulos contiene internamente sus cuatro capas lógicas (controlador, servicio, repositorio y entidad).



Compose y Dockerfile

En el compose se puede observar que el orquestador define dos servicios conectados a través de una red interna llamada `mexibanco`: la base de datos (`db`) y la aplicación (`app`). El servicio `db` utiliza una imagen de PostgreSQL. Por su parte, el servicio `app` cuenta con un bloque `depends_on` que condiciona su arranque hasta que la base de datos reporte un estado `service_healthy` como se vió en clase. Para verificar esta salud, ambos servicios implementan directivas `healthcheck` que ejecutan pruebas cíclicas.

Mientras que en el `dockerfile` se puede ver que en la etapa 1 (Construcción), se utiliza una imagen con el JDK de Java para descargar las dependencias con Maven, compilar el código fuente y empaquetarlo en un archivo `.jar` ejecutable. En la Etapa 2 (Ejecución), se utiliza una imagen mucho más reducida, a la cual únicamente se transfiere el archivo `.jar` generado en la etapa anterior para ejecutarlo en el puerto 8080.